

Normalización de la base de datos del reto apoyándose en recursos de IA generativa

La normalización de tablas en bases de datos es un proceso muy importante cuando se trabaja en el desarrollo de estas mismas. Apegándose a los procesos y formas de normalización de tablas obtenemos muchas ventajas cuando trabajamos con bases de datos. Por ejemplo, podemos reducir redundancia en tablas que tienen datos que se repiten en múltiples filas, se mantiene una mejor integridad de los datos y se evitan datos falsos o equivocados, se facilita el mantenimiento de la base de datos y en general se mejora el rendimiento de consultas.

Por otro lado, vivimos en una era donde las herramientas de lenguaje generativo de IA se han vuelto fundamentales para nuestro trabajo como desarrolladores de software, y este ámbito no es la excepción. Al hacer uso de estas herramientas durante procesos de normalización de tablas, obtenemos un segundo punto de vista, un agente inteligente que puede analizar nuestras tablas buscando problemas ocultos que violen las formas normales. Gracias al análisis de estas inteligencias artificiales, nosotros los desarrolladores podemos encontrar errores y solucionarlos de forma más rápida y efectiva. También son de gran utilidad al identificar posibilidades de simplificación y normalización. Todo esto potencia la efectividad del trabajo del desarrollador, y en vez de reemplazarlo, se vuelve un equipo mucho más poderoso.

En el caso de la base de datos de nuestro juego Z Attack, encontramos algunos cambios necesarios para normalizarla a 3ra forma normal. Para empezar, en nuestro SQL teníamos programada la tabla de usuarios, User, que tenía como elementos SMALLINT a role y status. Esto generaba una dependencia transitiva, porque el rol del usuario dependía del 1, 2 o 3 y no de la primary key: el id del usuario. Y era el mismo caso con status. Para solucionarlo creamos dos nuevas tablas, una para role y una para status, cada una con id y nombre. Y ahora, User simplemente toma el id del rol y el status que le corresponde y el valor (admin, jugador, activo o baneado) se determina en las tablas.

Teníamos el mismo problema en nuestra tabla Cards, que guardaba la rareza de la carta como un valor que se podía repetir muchas veces, generando redundancia. Para solucionarlo, creamos la tabla de Rarity con un id y el nombre de la rareza. Ahora Cards, toma el id y el valor se define en su propia tabla. Lo mismo con Card_type.

Por último, notamos que en nuestra tabla de Hero, teníamos un Hero.id_user, lo queataba a un héroe con un usuario específico, lo cual no tenía sentido, ya que los héroes son globales y no pertenecen a un único usuario. Eliminamos el id_user y

creamos la relación usuario-héroe con foreign keys en la tabla Run. Gracias a esto cambios logramos obtener una base de datos correctamente normalizada a 3FN.